The Setup — Gmail OAuth Per Brand

Each of your 100 brands connects their own Gmail account once. Your platform sends emails through their Gmail using Gmail API, not a bulk email service.

Brand clicks "Connect Gmail"

↓

Google OAuth consent screen

↓

Your app gets a refresh token for that brand's Gmail

↓

Stored encrypted in MongoDB (per brand)

↓

When brand sends campaign → your backend uses Gmail API

to send from their account

Cost — This Is Free

Gmail API is free up to 500 emails/day per connected account (Gmail's own sending limit, not a paywall). With 100 brands each sending to their own ~100 customers, you're nowhere near that limit per account.

ComponentCostGmail APIFreeGoogle OAuthFreeYour 10,000/month total volumeFree — well under Gmail's limits

The Catch — Gmail Sending Limits Per Account

This matters because it's per brand's Gmail, not your platform total:

Free Gmail account: 500 emails/day, 500 recipients/day

Google Workspace account: 2,000 emails/day

If a brand has 1,000 customers and wants to email all of them in one campaign, you need to batch and throttle — send in chunks across the day, not all at once.

pythonimport asyncio

```
async def send_campaign_throttled(recipients: list, brand_gmail_token: str, content: dict):
    BATCH_SIZE = 80  # stay safely under Gmail's per-hour soft limits
    DELAY_BETWEEN_BATCHES = 60  # seconds

    for i in range(0, len(recipients), BATCH_SIZE):
        batch = recipients[i:i + BATCH_SIZE]
        for recipient in batch:
            await send_single_email(brand_gmail_token, recipient, content)
            await asyncio.sleep(1)  # small gap between individual sends
        await asyncio.sleep(DELAY_BETWEEN_BATCHES)
```

Implementation — Gmail API Setup

1. Google Cloud Console

Create a project, enable Gmail API

Create OAuth 2.0 credentials (Web application type)

Add scope: https://www.googleapis.com/auth/gmail.send

2. OAuth Flow (FastAPI)

pythonfrom google_auth_oauthlib.flow import Flow

from googleapiclient.discovery import build

import base64

from email.mime.text import MIMEText

SCOPES = ['https://www.googleapis.com/auth/gmail.send']

@app.get("/connect-gmail")

async def connect_gmail(brand_id: str):

    flow = Flow.from_client_secrets_file(

        'client_secret.json',

        scopes=SCOPES,

        redirect_uri='https://yourapp.com/gmail-callback'

)

    auth_url, _ = flow.authorization_url(

        access_type='offline',

```
        prompt='consent',
        state=brand_id  # pass brand_id through OAuth flow
    )
    return {"auth_url": auth_url}


@app.get("/gmail-callback")
async def gmail_callback(code: str, state: str):
    brand_id = state
    flow = Flow.from_client_secrets_file(
        'client_secret.json',
        scopes=SCOPES,
        redirect_uri='https://yourapp.com/gmail-callback'
    )
    flow.fetch_token(code=code)
    credentials = flow.credentials

    # Store encrypted refresh token in MongoDB against this brand
    await db.brands.update_one(
        {"_id": brand_id},
        {"$set": {
            "gmail_refresh_token": encrypt(credentials.refresh_token),
            "gmail_connected": True,
```

```
            "gmail_email": get_user_email(credentials),
        }}
    )
    return {"status": "connected"}
```

3. Sending An Email

```
pythondef send_single_email(refresh_token: str, to_email: str, subject: str, html_body: str):
    credentials = build_credentials_from_refresh_token(decrypt(refresh_token))
    service = build('gmail', 'v1', credentials=credentials)

    message = MIMEText(html_body, 'html')
    message['to'] = to_email
    message['subject'] = subject

    raw = base64.urlsafe_b64encode(message.as_bytes()).decode()

    service.users().messages().send(
        userId='me',
        body={'raw': raw}
    ).execute()
```

Where Your Existing Copy Generation Plugs In

. Just route the output into this sender: